Information Web Retrieval and Analytics

Lecture 4: Scoring, Term Weighting and the Vector Space Model

Institute of Technology of Cambodia · Department of Applied Mathematics and Statistics

Today: Ranked retrieval

- Scoring documents
- Term frequency
- Collection statistics
- Weighting schemes
- Vector space scoring

Ranked retrieval

Problem with Boolean search: difficulty in query formulation

- Boolean queries
  - Terms + Boolean operators

- Most (non-expert) users are likely to have difficulty in writing Boolean queries.
  - What are the correct terms to use?
  - What do the operators mean and how to use them?

Problem with Boolean search: feast or famine

- Boolean logic is quite strict
- They can result in either too few (=0) or too many (1000s) results.
  - Q1: "Windows 10" **AND** login **AND** KB3081444 → 0 hits
  - Q2: "Windows 10" **OR** login **OR** KB3081444 → 377M hits
    - Also called "information overload"
- All the returned results are considered equally good by the search engine…

Problem with Boolean search: no differentiation

- **Good** for expert users with precise understanding of their needs and the collection.
  - Also good for applications: Applications can easily consume 1000s of results.

- **Not good** for the majority of users.
  - Most users don’t want to wade through 1000s of results.

Ranked retrieval

- **Free text queries**: The user's query is just one or more words in a human language.
- **Ranked results**: The results are ranked in the order of estimated relevance.
- Two separate choices, but a common combination.

Ranked retrieval

- All the users need to do is:
  - Write a free-text query and check the top k (≈ 10) results
    - If the results are good, the search is done.
    - Otherwise, repeat this process with a reformulated query.
- Simple and cost-effective
- However, the ranking algorithm must work (i.e., most relevant documents should be ranked as the top results.)

Scoring as the basis of ranked retrieval

How to rank the documents in the collection with respect to a query?

- Assign a score to each document
  - A number in [0, 1] which measures how well the query and the document match.
- Sort the documents based on the scores
  - Documents with score = 1
  - Documents with score = 0.99
  - …

1. Term frequency

Recap: binary term-document incidence matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	1	1	0	0	0	1	
BRUTUS	1	1	0	1	0	0	
CAESAR	1	1	0	1	1	1	
CALPURNIA	0	1	0	0	0	0	
CLEOPATRA	1	0	0	0	0	0	
MERCY	1	0	1	1	1	1	
WORSER	1	0	1	1	1	0	
...							

Each document is represented as a binary vector $\in \{0,1\}^{|V|}$.

Recap: binary term-document incidence matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	1	1	0	0	0	1	
BRUTUS	1	1	0	1	0	0	
CAESAR	1	1	0	1	1	1	
CALPURNIA	0	1	0	0	0	0	
CLEOPATRA	1	0	0	0	0	0	
MERCY	1	0	1	1	1	1	
WORSER	1	0	1	1	1	0	
...							

Each document is represented as a binary vector $\in \{0,1\}^{|V|}$.

1. Term frequency matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	157	73	0	0	0	1	
BRUTUS	4	157	0	2	0	0	
CAESAR	232	227	0	2	1	0	
CALPURNIA	0	10	0	0	0	0	
CLEOPATRA	57	0	0	0	0	0	
MERCY	2	0	3	8	5	8	
WORSER	2	0	1	1	1	5	
...							

Each document is now represented as a count vector $\in \mathbb{N}^{|V|}$.

1. Term frequency matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	157	73	0	0	0	1	
BRUTUS	4	157	0	2	0	0	
CAESAR	232	227	0	2	1	0	
CALPURNIA	0	10	0	0	0	0	
CLEOPATRA	57	0	0	0	0	0	
MERCY	2	0	3	8	5	8	
WORSER	2	0	1	1	1	5	
...							

Each document is now represented as a count vector $\in \mathbb{N}^{|V|}$.

Term frequency tf

- The term frequency $tf_{t,d}$ of term t in document d is defined as the number of times that t occurs in d.
- Shall we use $tf_{t,d}$ as it is?

- **Relevance** does not increase proportionally with **raw term frequency**
  - A document with 10 occurrences of the term is more relevant than a document with 1 occurrence. But not 10 times more relevant.

Log-frequency weighting scheme

- The log frequency weight of term *t* in *d* is

$$w_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{tf}_{t,d}, & \text{if } \mathrm{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

e.g. $0 \rightarrow 0$, $1 \rightarrow 1$, $2 \rightarrow 1.3$, $10 \rightarrow 2$, $1000 \rightarrow 4$, etc.

- Score of a document-query pair: **sum** over terms *t* in both *q* and *d*; 0 if no query term is in the document

Q = *Antony Cleopatra Calpurnia*, D = *Antony and Cleopatra*:
Score(Q, D) = $(1 + \log_{10} 157) + (1 + \log_{10} 57) + 0 = 3.20 + 2.76 + 0 =$ **5.96**

Formula: NUS CS3245 Information Retrieval, Lecture 7

2. Document frequency

2. Document frequency

- **Rare terms are more informative** than frequent terms
  - Given a query: **the emperor**, it is more important to match "emperor" than to match "the".
- We want…
  - **Lower weights** for **more common words** like *the, increase,* and *line*, and
  - **Higher weights** for **rarer ones** like *emperor*, and *arachnocentric*.
- This can be captured by the **inverse document frequency** (idf) weighting scheme.

idf weighting scheme

- df_t is the document frequency of t: the number of documents that contain t
  - df_t is an inverse measure of the informativeness of t
  - $df_t \leq N$ where N is the collection size.

- We define the idf (inverse document frequency) of t by

$$\text{idf}_t = \log_{10}(N/\text{df}_t)$$

  - We use log (N/df$_t$) instead of 1/df$_t$ to keep the value non-negative and dampen the effect of idf.

Formula: NUS CS3245 Information Retrieval, Lecture 7

Example: suppose N = 1 million

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

$$\text{idf}_t = \log_{10}(N/\text{df}_t)$$

There is **one idf value for each term** *t* in a collection.

3. tf-idf weighting

tf-idf weighting

- The tf-idf weight of a term is the product of its tf weight and its idf weight.

$$w_{t,d} = (1 + \log \mathrm{tf}_{t,d}) \times \log_{10}(N / \mathrm{df}_t)$$

- **Best known weighting scheme IR**
  - Note: the "-" in tf-idf is a hyphen, not a minus sign!
  - Alternative names: tf.idf, tf x idf
- **Increases** with the number of occurrences within a document
- **Increases** with the rarity of the term in the collection

Final ranking of documents for a query

$$\text{Score}(q,d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d}$$

- There are many variants
  - How “tf” is computed (with/without logs)
  - Whether the terms in the query are also weighted
  - …

Final ranking of documents for a query

- **tf-idf matrix**: binary → count → **weight** matrix

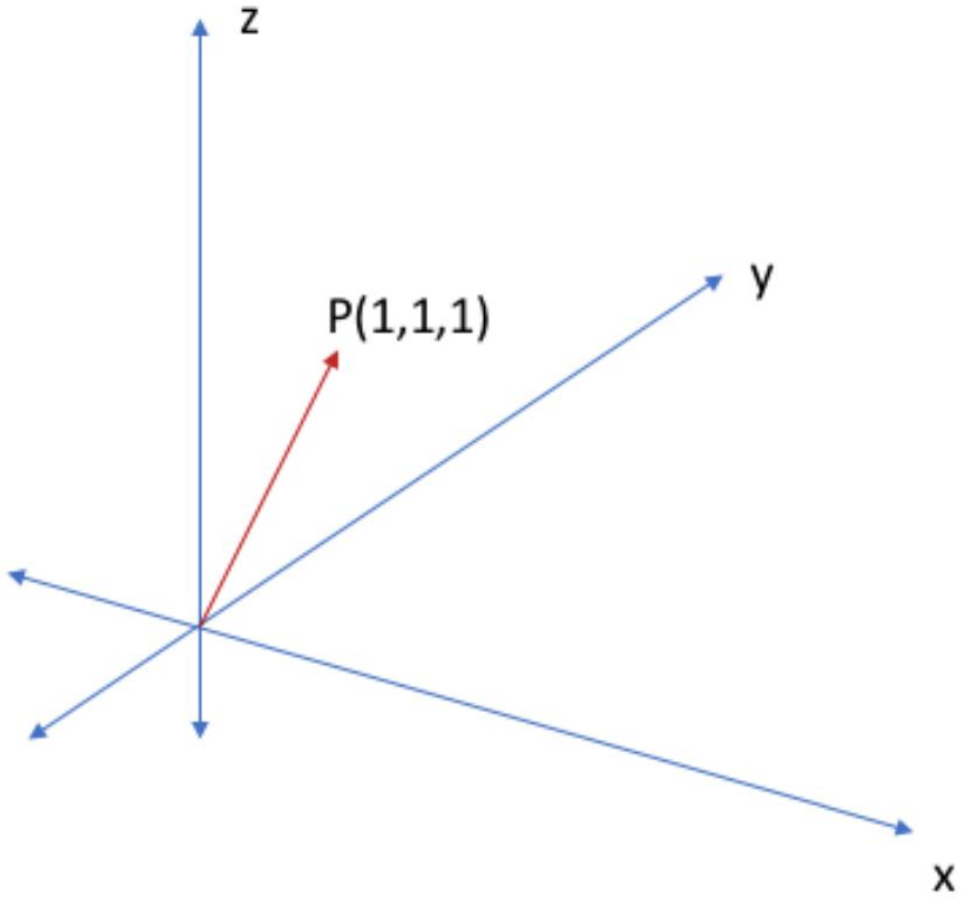
	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95

Let's say Q = *Antony Cleopatra Calpurnia*, D = *Antony and Cleopatra*
Score(Q, D) = 5.25 + 2.85 + 0 = **8.10**

4. The vector space model

Vector and vector space

- A 3-dimensional vector space with a vector P = (1, 1, 1)



Documents as vectors

- So we have a |V|-dimensional vector space
- Terms are axes of the space
- Documents are points or vectors in this space
- Very high-dimensional: tens of millions of dimensions when you apply this to a web search engine
- These are very sparse vectors - most entries are zero.

Queries as vectors

- Key idea 1: Do the same for queries: represent them as vectors in the space; they are "mini-documents"
- Key idea 2: Rank documents according to their **proximity** to the query in this space

	Q: Antony mercy		Antony and Cleopatra	Julius Caesar
Antony	2.45	Antony	5.25	3.18
Brutus	0	Brutus	1.21	6.1
Caesar	0	Caesar	8.59	2.54
Calpurnia	0	Calpurnia	0	1.54
Cleopatra	0	Cleopatra	2.85	0
mercy	1.21	mercy	1.51	0
worser	0	worser	1.37	0

Formalizing vector space proximity

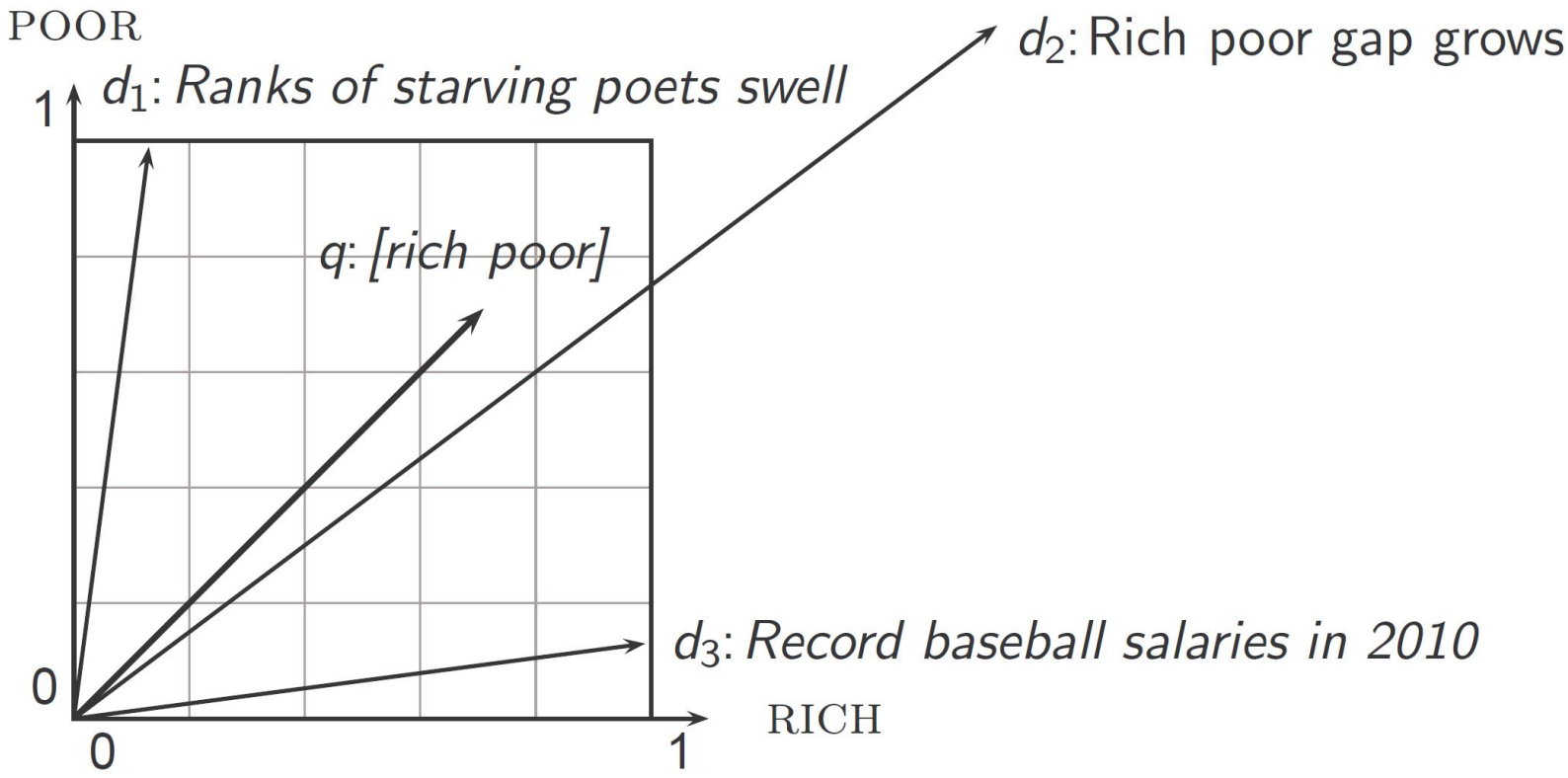
- First cut: distance between two points
  - (= distance between the end points of the two vectors)
- **Euclidean distance?**

$$d(\mathbf{p}, \mathbf{q}) = d(\mathbf{q}, \mathbf{p}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \cdots + (q_n - p_n)^2}$$

$$= \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}.$$

- Euclidean distance is a bad idea … because Euclidean distance is **large** for vectors of **different lengths**

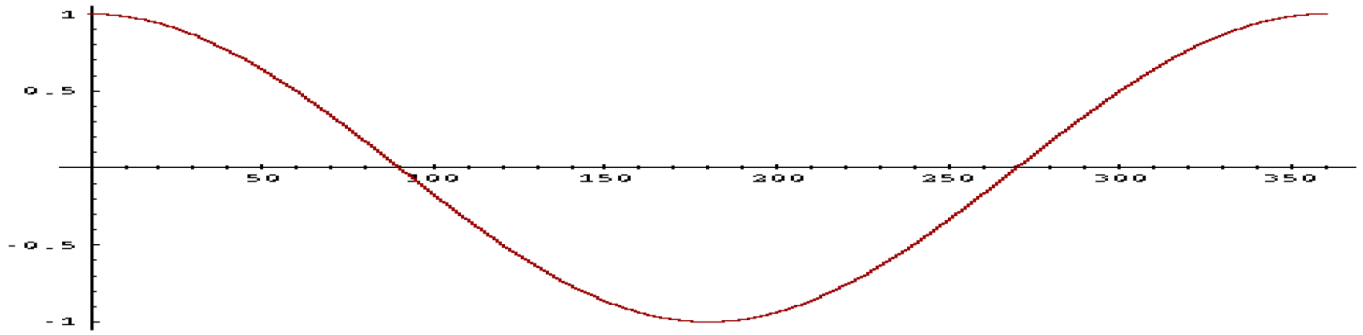
Why distance is a bad idea



The Euclidean distance of $\vec{q}$ and $\vec{d_2}$ is large although the distribution of terms in the query q and the distribution of terms in the document d_2 is very similar.

From angles to cosines

- The following two notions are equivalent.
  - Rank documents in decreasing order of the angle between query and document
  - Rank documents in increasing order of cosine(query, document)
- Cosine is a monotonically decreasing function for the interval [0°, 180°]



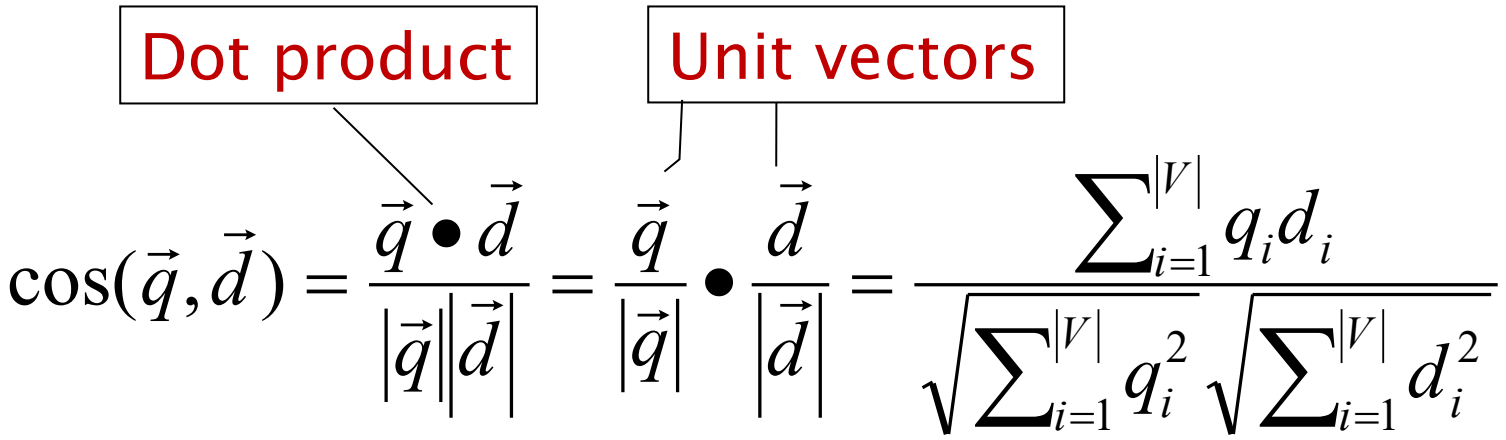
Length normalization

- A vector can be (length-) normalized by dividing each of its components by its length – for this we use the L_2 norm:

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- Dividing a vector by its L_2 norm makes it a unit (length) vector (on surface of unit hypersphere)
- Effect on the two documents d and d′ (d appended to itself) from earlier slide: they have identical vectors after length-normalization.
  - Long and short documents now have comparable weights

cosine(query, document)



q_i is the weight of term i in the query
d_i is the weight of term i in the document

cos(q,d) is the cosine similarity of q and d … or, equivalently, the cosine of the angle between q and d.

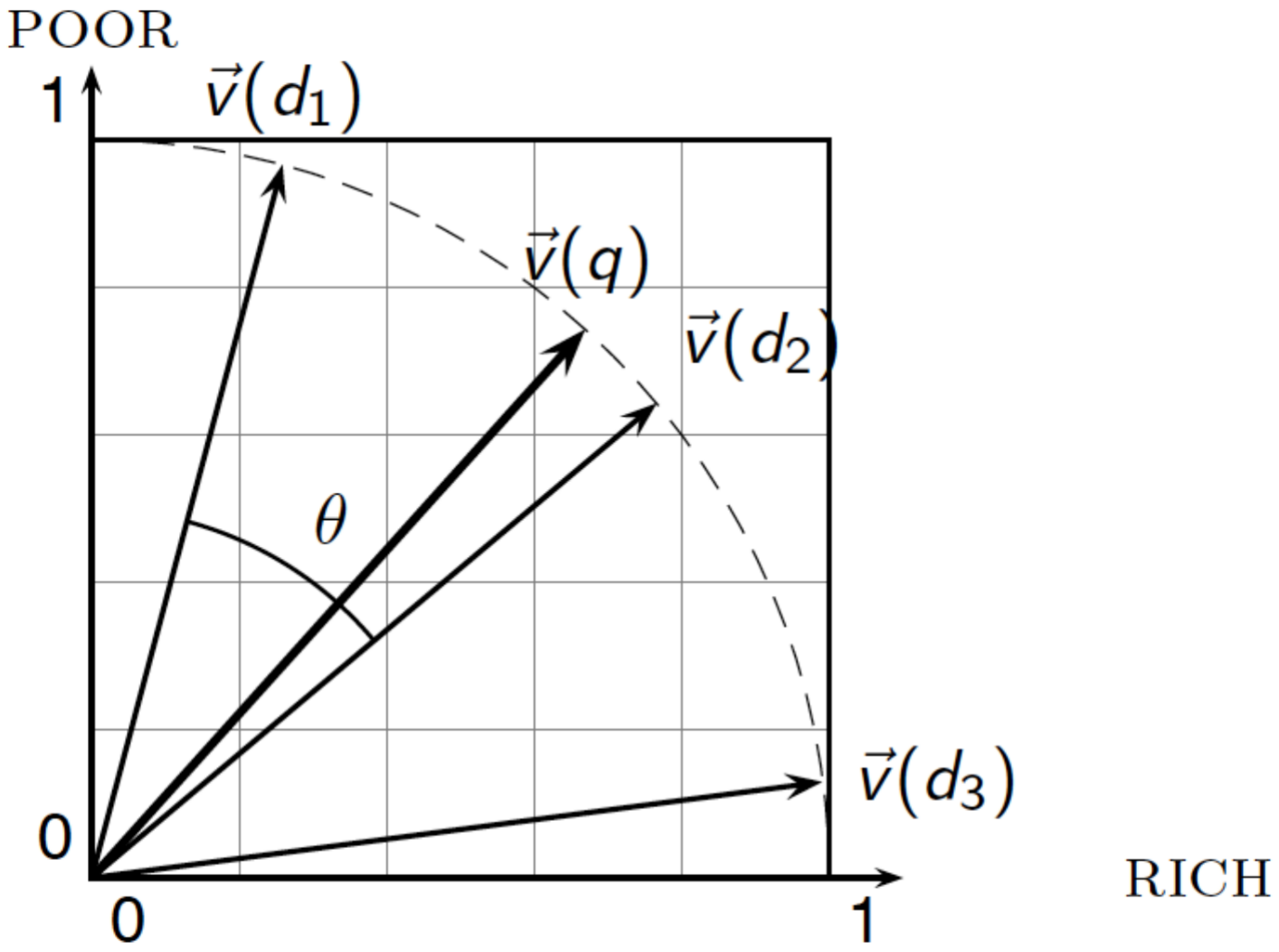
Cosine for length-normalized vectors

- For length-normalized vectors, cosine similarity is simply the dot product (or scalar product):

$$\cos(\vec{q},\vec{d}) = \vec{q} \bullet \vec{d} = \sum_{i=1}^{|V|} q_i d_i$$

for q, d length-normalized.

Cosine similarity illustrated



Cosine similarity example

- How similar are these documents vs the query:

affection jealous

term	Doc 1	Doc 2	Q
affection	115	58	1
jealous	10	7	1

Term frequencies

Note: To simplify this example, we do not do idf weighting and consider only two terms.

Cosine similarity example

Log term frequency weighting

term	Doc 1	Doc 2	Q
affection	3.06	2.76	1
jealous	2.00	1.85	1

After length normalization

term	Doc 1	Doc 2	Q
affection	0.84	0.83	0.71
jealous	0.55	0.56	0.71

$\cos(\text{Doc 1}, Q) \approx 0.84 \times 0.71 + 0.55 \times 0.71 \approx 0.99$

$\cos(\text{Doc 2}, Q) \approx 0.99$

Cosine similarity amongst 3 documents

How similar are the novels
SaS: *Sense and Sensibility*
PaP: *Pride and Prejudice*, and
WH: *Wuthering Heights*?

term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Term frequencies (counts)

Note: to simplify this example, we don't do idf weighting.

3 documents example contd.

Log frequency weighting

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

After length normalization

term	SaS	PaP	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0	0.405
wuthering	0	0	0.588

dot(SaS,PaP) ≈ 12.1
dot(SaS,WH) ≈ 13.4
dot(PaP,WH) ≈ 10.1

cos(SaS,PaP) ≈ 0.94
cos(SaS,WH) ≈ 0.79
cos(PaP,WH) ≈ 0.69

Why do dot product and cosine disagree about which pair is most similar?

5. Computing scores and weighting variants

tf-idf weighting has many variants

Term frequency		Document frequency		Normalization	
n (natural)	$\mathrm{tf}_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(\mathrm{tf}_{t,d})$	t (idf)	$\log \frac{N}{\mathrm{df}_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2+w_2^2+\ldots+w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times \mathrm{tf}_{t,d}}{\max_t(\mathrm{tf}_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N-df_t}{df_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases} 1 & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^{\alpha}$, $\alpha < 1$
L (log ave)	$\frac{1+\log(tf_{t,d})}{1+\log(ave_{t \in d}(tf_{t,d}))}$				

Best known combination of weighting options

tf-idf Nano-example

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

tf-idf Nano-example

Query: sweet love

$|q| = \sqrt{}$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized
sweet	**1**					
nurse	0					
love	**1**					
how	0					
sorrow	0					
is	0					

tf-idf Nano-example

Query: sweet love

$$|q| = \sqrt{(.125^2 + .301^2)} = .325$$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized
sweet	**1**	1	3	0.125	0.125	0.383
nurse	0					
love	**1**	1	2	0.301	0.301	0.924
how	0					
sorrow	0					
is	0					

tf-idf Nano-example

Doc 2: Sweet sorrow

$$|q| = \sqrt{}$$

Word	Count	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	× q
sweet	**1**						
nurse	0						
love	0						
how	0						
sorrow	**1**						
is	0						

tf-idf Nano-example

Doc 2: Sweet sorrow

$$|q| = \sqrt{(.125^2 + .602^2)} = .615$$

Word	Cnt	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	× q
sweet	**1**	1	3	0.125	.125	.203	0.0779
nurse	0						
love	0						
how	0						
sorrow	**1**	1	1	.602	0.602	.979	0
is	0						

tf-idf Nano-example

Query: sweet love . **Doc 2**: Sweet sorrow = 0.0779

Word	Cnt	tf $1+\log_{10}(c)$	df	idf $\log_{10}N/df)$	tf-idf tf x idf	normalized	× q
sweet	**1**	1	3	0.125	.125	.203	**0.0779**
nurse	0						
love	0						
how	0						
sorrow	**1**	1	1	.602	0.602	.979	0
is	0						

tf-idf Nano-example

Query						
word	**cnt**	**tf**	**df**	**idf**	**tf-idf**	**n'lized** = tf-idf/$\|q\|$
sweet	1	1	3	0.125	0.125	0.383
nurse	0	0	2	0.301	0	0
love	1	1	2	0.301	0.301	0.924
how	0	0	1	0.602	0	0
sorrow	0	0	1	0.602	0	0
is	0	0	1	0.602	0	0
$\|q\| = \sqrt{.125^2 + .301^2} = .326$						

	Document 1					Document 2				
word	**cnt**	**tf**	**tf-idf**	**n'lized**	**× q**	**cnt**	**tf**	**tf-idf**	**n'lized**	**×q**
sweet	2	1.301	0.163	0.357	**0.137**	1	1.000	0.125	0.203	**0.0779**
nurse	1	1.000	0.301	0.661	0	0	0	0	0	0
love	1	1.000	0.301	0.661	**0.610**	0	0	0	0	**0**
how	0	0	0	0	0	0	0	0	0	0
sorrow	0	0	0	0	0	1	1.000	0.602	0.979	0
is	0	0	0	0	0	0	0	0	0	0
$\|d_1\| = \sqrt{.163^2 + .301^2 + .301^2} = .456$						$\|d_2\| = \sqrt{.125^2 + .602^2} = .615$				
Cosine: $\sum$ of column:					**0.747**	Cosine: $\sum$ of column:				**0.0779**

tf-idf Nano-example

Final cosine

(details in chapter)

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

score(q,d1) = 0.747

score(q,d2) = 0.0779

- d1 has both terms, including 2 instances of *sweet*
- d2 is missing one of the terms

tf-idf example: lnc.ltc

Document: *car insurance auto insurance*
Query: *best car insurance*

(N = 1000000)

Term	Document				Query						Prod
	tf-raw	tf-wt	wt	n'lize	tf-raw	tf-wt	df	idf	wt	n'lize	
auto	1	1	1	0.52	0	0	5000	2.3	0	0	0
best	0	0	0	0	1	1	50000	1.3	1.3	0.34	0
car	1	1	1	0.52	1	1	10000	2.0	2.0	0.52	0.27
insurance	2	1.3	1.3	0.68	1	1	1000	3.0	3.0	0.78	0.53

Doc length $= \sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$

Score = 0+0+0.27+0.53 = 0.8

Summary and algorithm: vector space ranking

1. Represent the query as a weighted *tf-idf* vector
2. Represent each document as a weighted *tf-idf* vector
3. Compute the cosine similarity score for the query vector and each document vector
4. Rank documents with respect to the query by score
5. Return the top K (e.g., $K = 10$) to the user

Thank you!